



FCN & DeepLabv3

📅 날짜	@2025년 4월 3일
≡ 분야	분석
≡ 주차	

FCN

Abstract

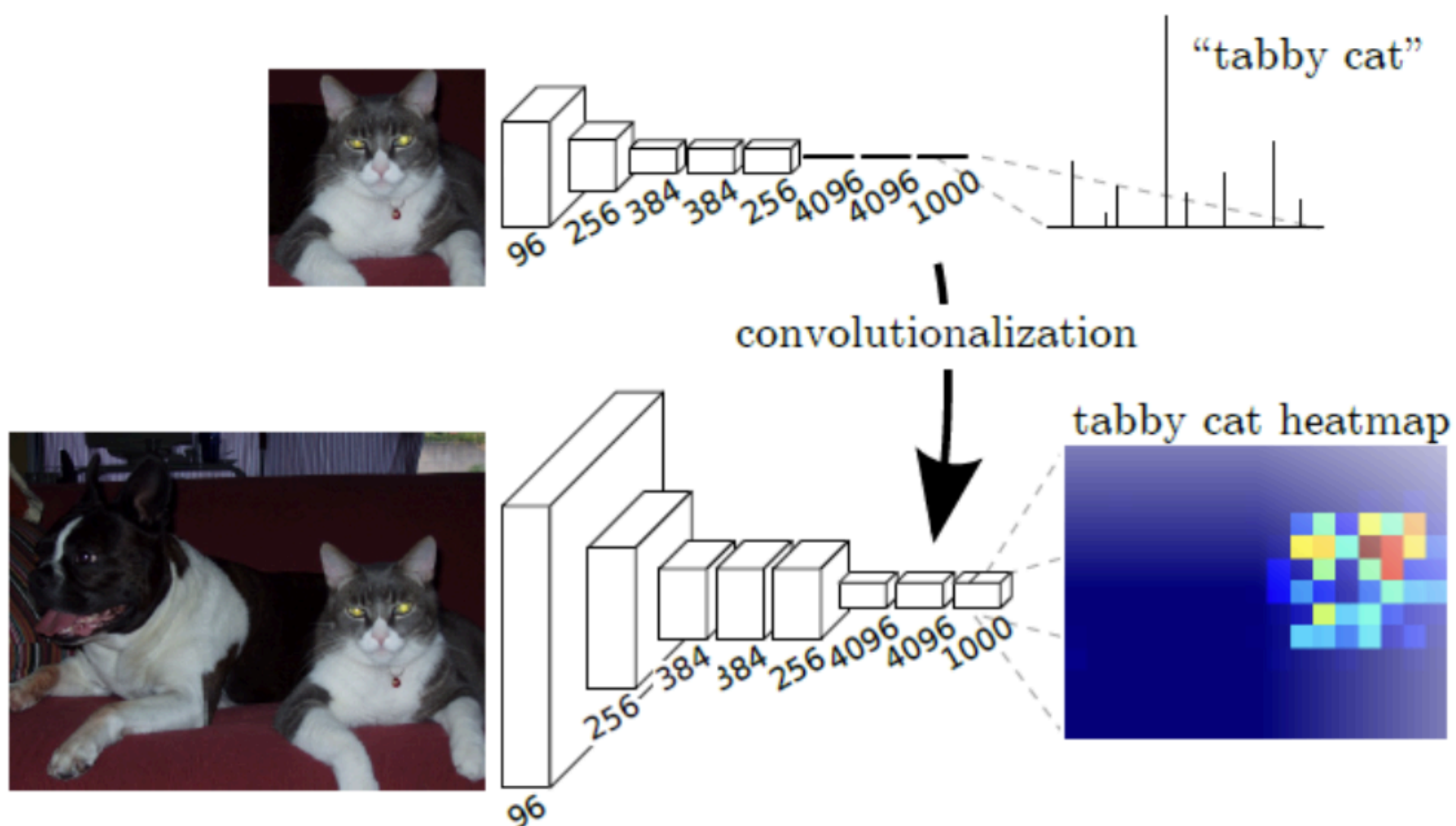
- Convolutional Network의 구조를 end-to-end, pixels-to-pixels 방식으로 학습시켜 semantic segmentation 분야에서 SOTA 달성
- 임의의 사이즈의 이미지를 인풋으로 넣었을 때, **동일한 사이즈의 아웃풋이 나오도록 fully convolutional network**를 구성하여 사용한다.
- 기존 classification network인 AlexNet, VGGnet 등을 **fine tuning**하여 사용하며 **coarse(전반적인) 정보와 fine(세밀한) 정보를 혼합**시킨다.

Introduction

- Convnet의 유용성은 classification 뿐만 아니라 object detection, par and key point prediction, local correspondence로 점차 발전해왔다.
- 이처럼 점점 더 세부적인 정보를 예측하게 되면서 픽셀 단위의 예측까지 관심을 가지게 되었는데 이를 segmentation이라 한다.
- 본 논문은 픽셀 단위의 예측을 위한 최초의 **end-to-end 방식이자 지도 사전 학습**을 활용한다.
- 기존의 네트워크에서는 dense한 형태의 아웃풋이 나오나 본 논문에서는 동일한 사이즈의 아웃풋을 리턴하기 위해 **upsampling 기법**을 활용한다. 따라서, **이미지에 대한 전처리나 후처리가 필요하지 않다.**
- skip 구조**를 활용하여 이미지의 **전반적인 정보와 세부 정보를 혼합**시킨다.

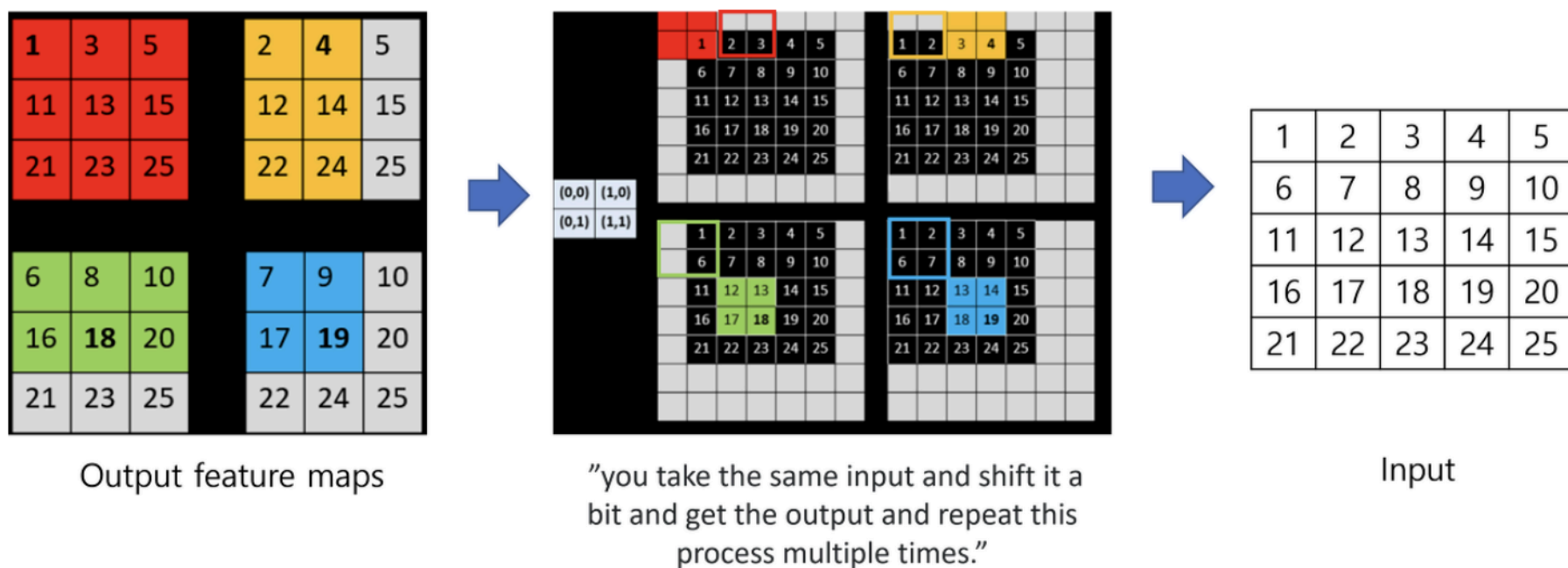
Fully convolutional networks

Adapting classifier for dense prediction



- 기존의 네트워크(LeNet, AlexNet 등)들은 아키텍처의 끝 부분에 **FC layer**가 있어 공간 정보를 담지 못한채 **output**이 나오게 된다.
- Segmentation의 경우 공간 정보를 유지하여 픽셀 단위로 예측해야 하므로 이러한 문제를 **1x1 Conv**를 활용하여 **spatial dimension**이 나오도록 만든다.
- 또한, 기존 CNN의 경우 input으로 패치를 받는 반면 FCN에서는 **전체 이미지를 받아 효율적인 계산량**을 가지게 된다.

Shift-and-stitch is filter rarefaction



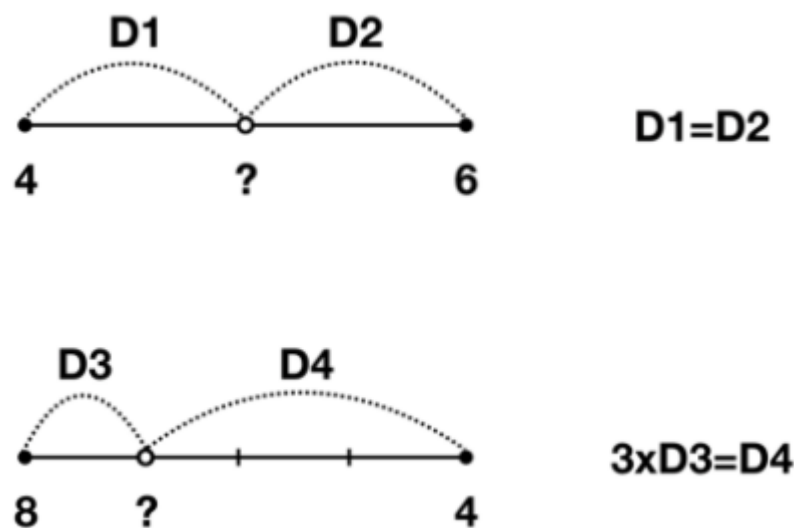
- coarse한 output으로부터 dense prediction을 얻는 방법으로 OverFeat 논문에서 쓰였던 **Shift-and-stitch** 방법 소개
- Pooling 커널의 사이즈나 stride에 변화를 주는 경우 trade-off가 존재한다. **Downsampling**을 작게 할 경우 **receptive field**가 작아 연산량은 많으나 **finer**한 정보를 얻게 된다.
- Shift-and-stitch trick을 쓰면 커널의 사이즈나 stride의 감소 없이도 패딩 방식을 다양하게 하여 dense prediction을 만들 수 있지만, **receptive field**가 작을수록 **finer**한 정보를 얻기에 제한적일 수 있다.
- 본 논문에서는 결과적으로 이 방식 대신 **skip layer fusion**을 사용한다.

Upsampling is backwards strided convolution

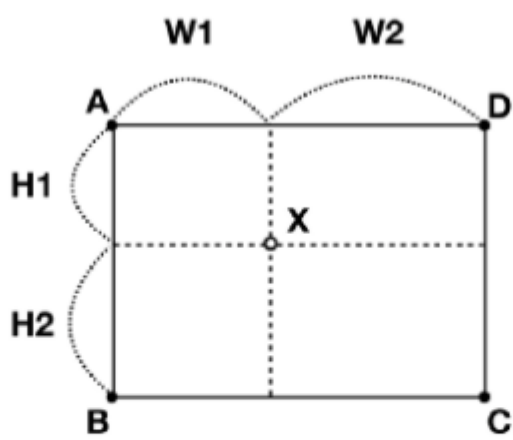
Upsampling을 하여 dense map을 얻는 방법으로 **2가지 방법**이 더 소개된다.

1)Interpolation

주어진 값들 사이의 값을 linear하게 추정한다.



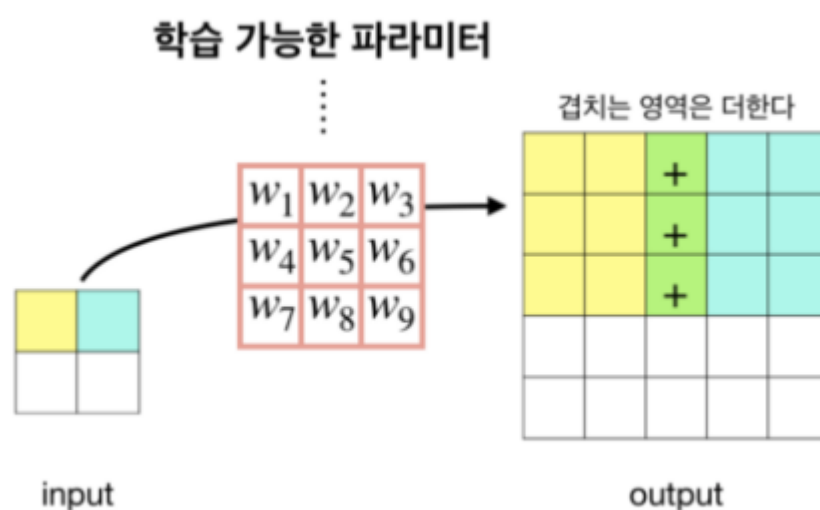
1D의 경우부터 살펴보면 위의 값은 5, 아래의 값은 7이라는 것을 쉽게 알 수 있다.



$$X = \left(A \frac{H2}{H1 + H2} + B \frac{H1}{H1 + H2} \right) \frac{W2}{W1 + W2} + \left(D \frac{H2}{H1 + H2} + C \frac{H1}{H1 + H2} \right) \frac{W1}{W1 + W2}$$

이를 2D로 확장하면 x,y 좌표를 각각 계산하면 된다. 이와 같은 방법으로 저해상도의 이미지를 고해상도의 이미지로 확장할 때 **비어있는 중간 값들을 유추**하여 채워줄 수 있다.

2)Deconvolution

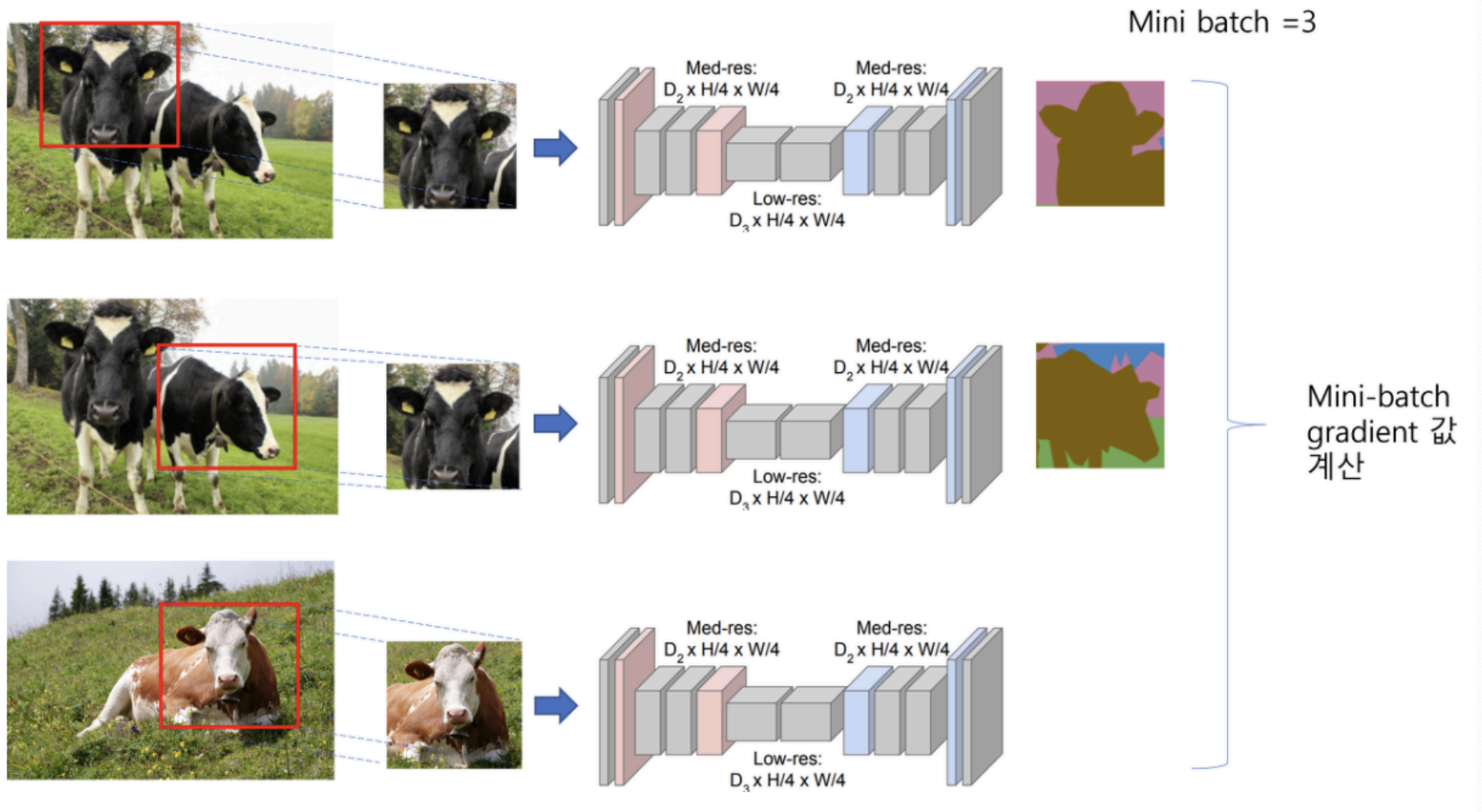


Deconvolution은 Convolution의 역연산이다. 위의 그림과 같이 **겹치는 영역은 더해서** 구해주면 **Upsampling된 output**을 얻을 수 있다.

하지만 2가지 방식 전부 이미 크게 줄어든 상태의 **feature map**으로부터 추정을 하는 형태기 때문에 여전히 정보손실은 클 수 밖에 없다. 따라서 본 논문에서는 **skip architecture**까지 적용해준다.

Patchwise training is loss sampling

<Patch wise training>



- **patchwise** 방식으로 학습할 경우 패치들 간에 겹치는 부분이 많아 불필요한 연산량이 많아질 수 있다. 반면, **전체 이미지를 사용할 경우** 사이즈가 큰 경우 **GPU 메모리를 많이 차지하여 충분한 미니배치를 잡기 어렵다.**
- **패치** 방식으로 학습을 시켜본 결과 더 빠르거나 loss가 잘 수렴하는 등의 **장점은 없어** 전체 이미지를 입력으로 받는 방식을 쓴다고 한다.

Segmentation Architecture

- FCN은 Downsampling을 하는 Encoder 부분과 Upsampling을 하는 Decoder 부분으로 구성되어 있다.
- Downsampling은 기존 CNN 구조와 동일하여 사전학습된 모델을 사용한다.
- loss function은 각 픽셀마다 multinomial logistic loss가 사용되었다.

From classifier to dense FCN

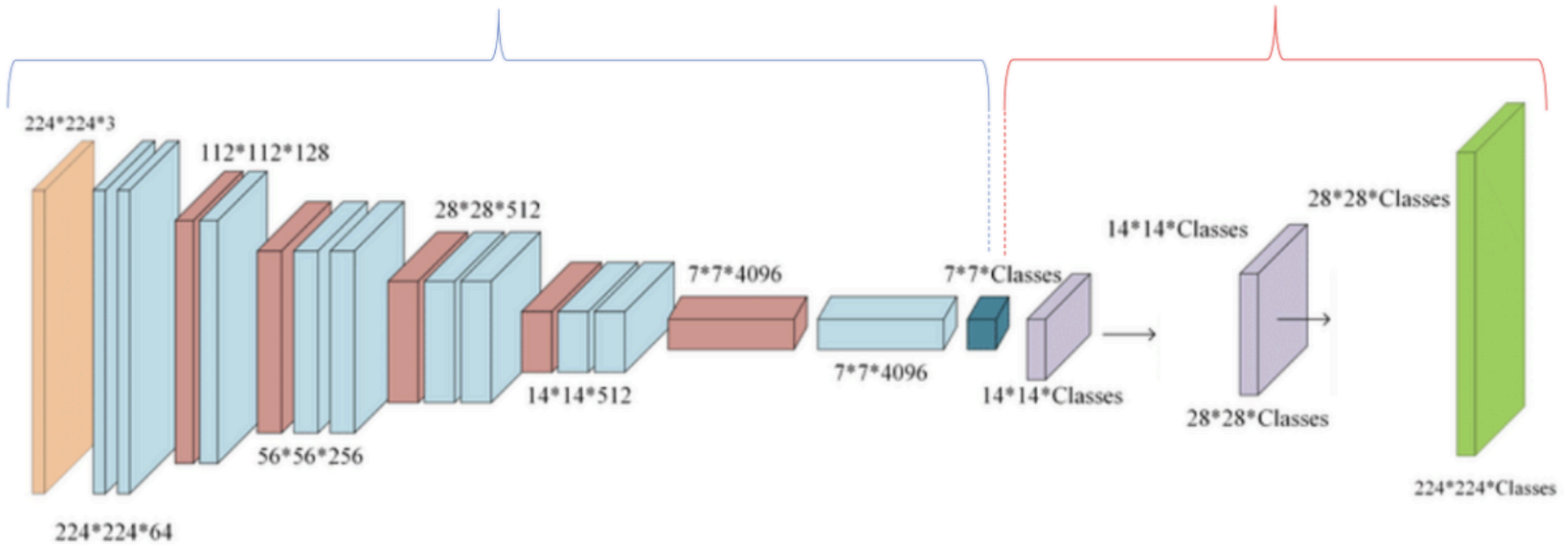
- 앞서 보았듯이 Classifier는 기존 모델들의 **FC층을 Convolution으로** 바꿔주어 사용한다.
- AlexNet, VGG, GoogLeNet을 사용한 결과 **VGG가 가장 좋은 성능**을 보였다.
- VGG와 GoogLeNet은 분류 task에서는 성능이 비슷함에도 불구하고 segmentation을 할 때는 GoogLeNet이 VGG를 따라오지 못했다.

Combining what and where

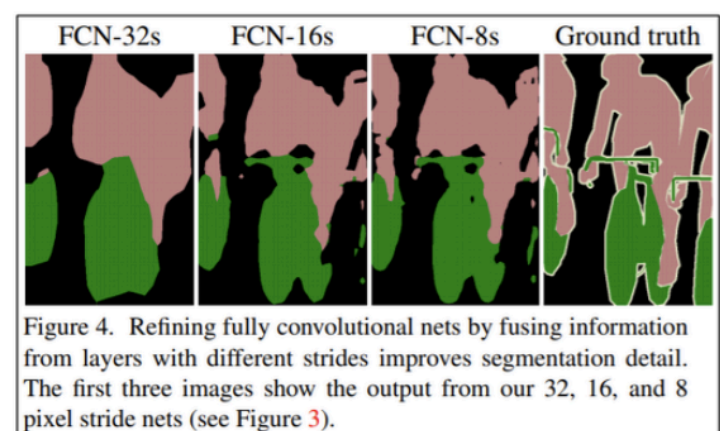
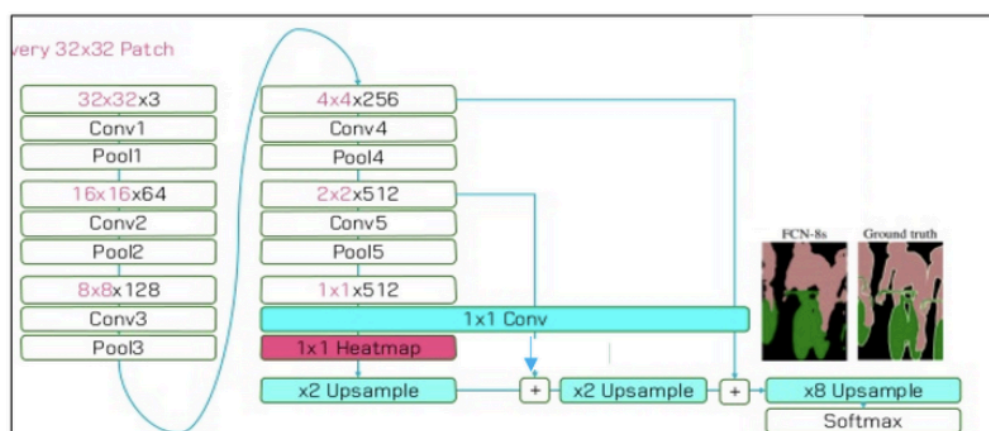
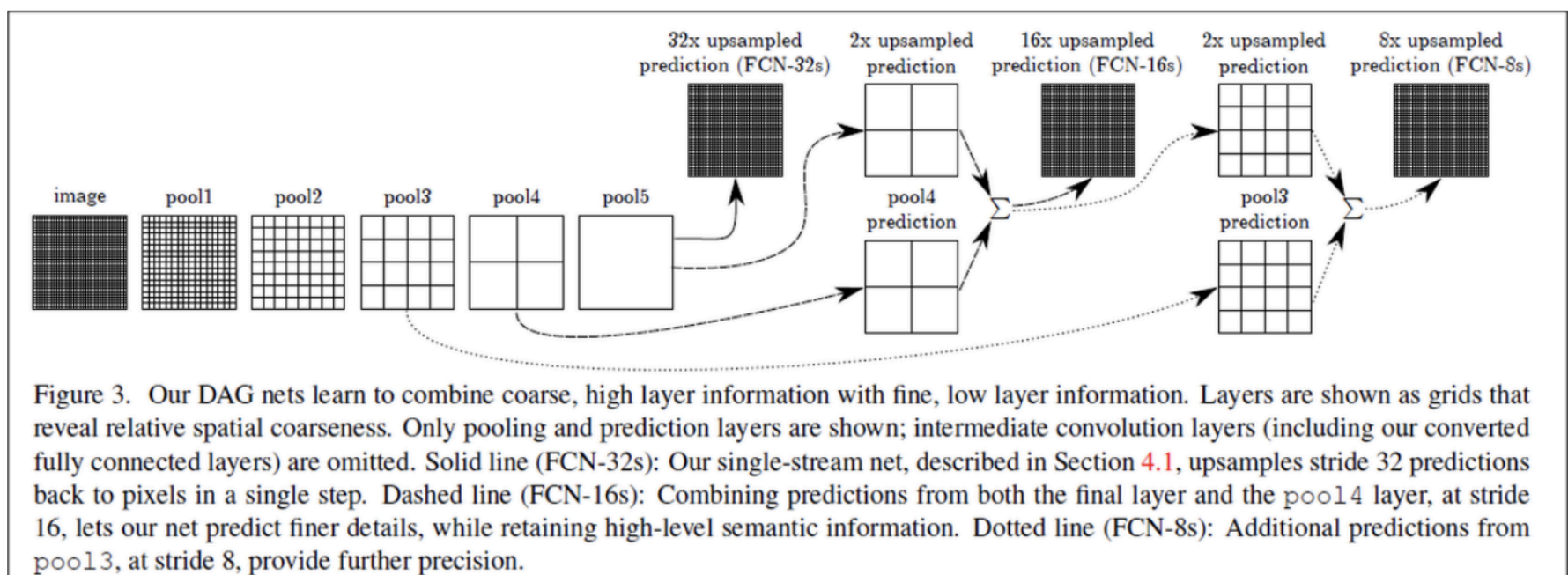
지금까지의 구조를 살펴보면 다음과 같고 해당 구조는 segmentation에서 충분한 성능을 보이지 못해 **skip architecture**를 제시한다.

Convolution = Down-sampling

Deconvolution = Up-sampling



- 논문에서 나온 FCN-32는 32x32의 이미지를 input으로 하고 1x1으로 feature map을 뽑은 후 다시 32x32로 upsampling한 결과를 의미한다.
- 이와 같이 **Pooling** 과정을 많이 거치는 경우 정보손실이 크기 때문에 upsampling을 하여 이미지의 사이즈를 맞춰준다 하더라도 **fine information**이 부족하여 segmentation에서 좋은 성능을 보이지 어렵다.
- 따라서 pooling을 덜 거친 **중간 단계의 feature map**들을 활용하자는 것이 skip architecture의 핵심이다.



이미지의 좌측 하단 그림을 통해 FCN-16s를 설명하면 다음과 같다.

1. **Pool5**를 통해 생성된 1x1 feature map을 **2배로 upsampling**한다.
2. **Pool4**에서 생성된 2x2x512 feature map에서 **1x1x512 필터**를 적용한다.
3. 위의 두 과정에서 연산된 결과를 **element-wise**로 더해준 후, **16배로 upsampling**하여 32x32로 만들어준다.

DeepLabv3

Introduction

두가지 난관

Semantic segmentation을 수행하는 Deep Convolutional Neural Networks(DCNNs)에 있어 다음과 같은 두가지 어려움이 존재한다.

1. 연속적인 pooling 작업 혹은 convolution strides 조절에 의한 정보 손실

두 작업은 굉장히 추상인 특성을 DCNNs에게 학습시키는데 사용할 수 있지만 세부적인 정보를 필요로 하는 dense prediction에 방해가 될 수 있다. 본 연구에서는 이 문제점을 해결하기 위해 atrous convolution을 사용했다. 확대된 convolution이라고도 불리는 atrous convolution은 기존 ImageNet으로 훈련된 모델의 마지막 몇개의 layer들을 제거하고 제거된 filter layer들을 upsampling하여 사용하는 방법을 일컫는다. 이는 기존 filter의 weight에 구멍을 뚫어 사용하는 것과 같다. 이를 통해 추가적인 매개변수들을 사용하지 않고 추론 결과의 해상도를 조절할 수 있다.

2. 제각각의 객체 크기

본 연구에서는 지금까지 이를 해결하기 위한 제시된 여러가지 방안 중 네가지를 살펴보도록 한다.

- **Image Pyramid**각기 다른 크기의 객체에 Image Pyramid를 사용하여 각기 다른 크기의 feature map을 얻어낸다.
- **Encoder-Decoder**Encoder에서 각기 다른 크기의 feature map을 얻어내고, 이것들의 해상도를 decoder에서 부분적으로 개선시킨다.
- **Extra modules**더 넓은 범위를 나타내는 정보를 얻기 위해 추가적인 layer를 쌓는다.
- **Spatial Pyramid Pooling**다양한 크기와 비율을 가진 pooling layer들을 적용해 다양한 scale의 객체들을 얻는다.

본 연구에서는 pooling 범위, 정보가 나타내는 넓이를 넓힐 수 있는 atrous convolution에 대해 cascaded modules과 spatial pyramid pooling에 대하여 적용한다. 특히 본 연구에서 사용한 atrous convolution은 다양한 비율과 batch normalization을 이용한다. 이러한 다양성은 학습에 있어 중요하다 생각한다. 또한, atrous convolution을 평행적 그리고 수직적, 두가지 방법으로 적용한다. 또한, 3×3 convolution에 대하여 높은 비율로 atrous convolution을 적용할 경우 발생하는 문제에 대해서도 다룬다.

추가 정보

Atrous Convolution

참고 : <https://better-tomorrow.tistory.com/m/entry/Atrous-Convolution>

Related Work

Semantic segmentation에서는 전역적인 정보와 지역적인 정보 간의 상호작용이 중요하다. 본 연구에서는 지역적인 정보를 얻기 위한 네가지의 Fully Convolutional Networks(FCNs)를 소개한다.

1. Image pyramid

같은 모델 즉, 같은 weight에 각기 다른 크기의 정보를 가지는(multi-scale) 입력을 입력하게 되면 출력 정보가 나타내는 scale이 다르다. 크기가 큰 객체의 경우 세부적인 정보가 보존되는 반면, 작은 객체의 경우 세부적인 정보가 생략된다. Farabet은 Laplacian pyramid를 이용해서 하나의 DCCN에 각기 다른 크기의 input을 입력한다. 도출되는 각기 다른 크기의 결과값을 통합한다. 각 층의 결과값, 점점 줄어드는 크기의 결과값을 병합한 후 최종적으로 병합하는 것이다. 하지만, 이 방법은 GPU memory의 부족으로 인해, scale up&down 수행이 원활하지 않다는 단점을 가진다.

2. Encoder-decoder

Encoder-decoder 모델은 encoder와 decoder 부분으로 나뉜다. Encoder는 점진적으로 input의 정보가 담긴 feature map의 크기를 줄여나간다. 이에 encoder가 깊을수록 더 넓은 범위(long range)의 정보를 담아낼 수 있다. Decoder은 deconvolution 등의 방법을 통해 점진적으로 생략된 세부 정보와 줄어든 크기를 복구한다. 이러한 encoder-decoder 모델은 object detection에 사용될 수 있다.

3. Context module

이 모델은(DCNNs) 넓은 범위의(long range) 정보를 담아내기(encode)위해 추가적인 모듈들을 수직적으로(in cascade) 추가한다. DenseCRF를 추가하는 것이 대표적인 예시이다. CRF와 DCNN이 조화롭게 학습되기 위해서 출력되는 feature map에 몇개의 convolution 연산을 추가적으로 진행시킨다.

4. Spatial pyramid pooling

Spatial pyramid pooling을 사용하여 다양한 범위의 정보를 얻는다. ParseNet은 image-level의 전역적인 정보를 얻어내고, DeepLabv2에서는 서로 다른 비율의 atrous convolution을 병렬적으로 진행해 다양한 범위의 정보를 얻어낸다. 최근에는 Pyramid scene PArsing

Net(PSP)가 spatial pooling을 다양한 크기로 진행하여 좋은 성과를 얻어냈다. 이러한 spatial pyramid pooling은 object detection에 이용될 수 있다.

본 연구에서는 context module 과 spatial pyramid pooling을 위해 atrous convolution을 사용한다. 자세한 구조는 다음과 같다. ResNet의 마지막 block을 여러번 복제해 마지막 부분에 수직으로 쌓는다. 또한, atrous convolution을 평행으로 여러개 배치한 ASPP 모듈을 배치하여 이용한다. 이렇게 수직으로 쌓은 모듈들에는 belief map이 아니라 feature map을 입력해준다는 점을 주의해야 한다. 또한, batch normalization과 함께 사용해야 좋은 결과를 얻을 수 있다.

Methods

이번 장에서는 dense feature를 얻어내기 위해 atrous convolution을 어떻게 사용했는지에 대해 다루고, atrous convolution을 수직, 수평으로 배치한 모듈에 대해 다룬다.

Atrous Convolution for Dense Feature Extraction

DCNNs는 fully convolutional한 방법을 사용해서 효율적으로 semantic segmentation을 수행한다. 그러나 반복되는 max-pooling 작업과 stride는 결과의 해상도를 낮춘다. Deconvolution 계층들이 낮아진 해상도를 높일 수 있지만 본 연구에서는 atrous convolution을 사용했다.

2차원의 입력 x , 출력 y , 필터 w 를 생각할 때, 위치 i 에 대하여 다음과 같은 연산이 수행된다. r 을 조절하며 filter의 시야(field-of-view)를 넓힐 수 있다.

$y[i] = \sum_k x[i+r \times k]w[k]$ (r 은 atrous rate임과 동시에 stride에 해당한다. 일반적인 convolution은 r 이 1이다.)

Atrous convolution을 이용하면 출력 이미지의 해상도를 직접적으로 조절할 수 있다. 여기서 output stride는 입력 이미지의 해상도와 출력 해상도의 비율을 의미한다. DCNNs의 입력 이미지와 fully-connected layer, global pooling을 적용하지 전 결과 이미지를 비교하면, 결과 이미지가 32배 작다는 것 즉, output stride가 32라는 것을 알 수 있다. 만약 출력의 공간적 밀집도(spatial density)를 2배로 높이고 싶다면, down sampling (decimation)을 피하기 위해 마지막 pooling 혹은 convolution의 stride를 1로 설정하고 하위의 모든 convolutional layer들을 r 이 2인 atrous convolution으로 대체하면 된다. 이러한 방식을 사용하면 추가적인 파라미터를 사용하지 않고도 출력의 밀집도를 높일 수 있다.

Going Deeper with Atrous Convolution

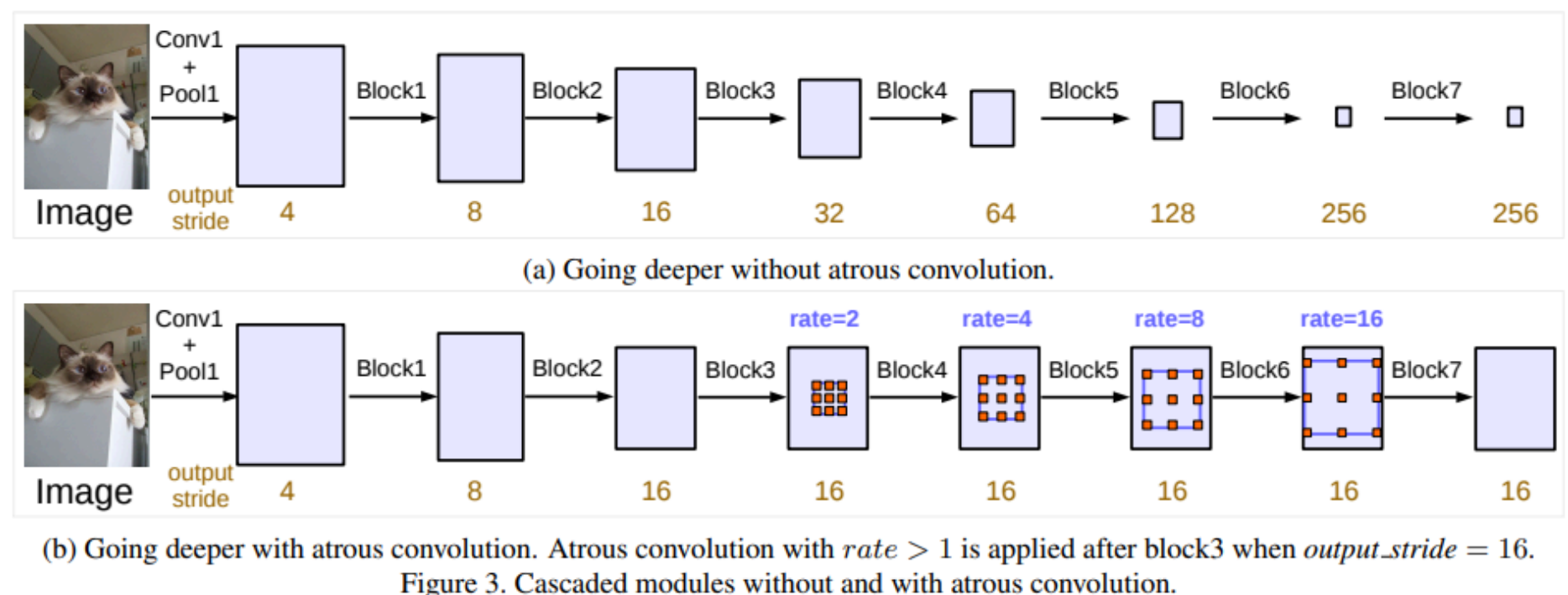


사진 출처 : 본 논문 (<https://doi.org/10.48550/arXiv.1706.05587>)

먼저 atrous convolution을 수직으로 적용한 부분을 살펴보겠다. ResNet의 마지막 모듈을 여러 개 복사한 후 DCNNs의 끝 부분에 수직으로 배치하였다. 해당 모듈들에는 각각 3×3 convolution들이 포함되어 있고, 복사하여 덧붙인 모듈 중 마지막 모듈을 제외하고는 마지막 convolution의 stride는 2이다. 이와 같은 배열은 더 넓은 범위의 정보를 얻기 편하게 하는 데에 있다. 그러나 semantic segmentation에서는 이와 같은 작업이 세세한 정보들을 삭제하기에 적합하지 않다. 따라서 atrous convolution을 사용해야 한다. 이때, 가장 바람직한 output stride는 16이다.

Atrous Spatial Pyramid Pooling (ASPP)

각기 다른 atrous rate를 가지고 ASPP를 진행하면 multi-scale한 정보를 효율적으로 얻어낼 수 있다. 그러나 rate이 커질수록 유효한 가중치들이 감소하는 것을 관찰할 수 있었다. 65×65 의 입력에 3×3 의 필터를 이용해서 atrous convolution을 진행하였을 때, atrous rate이 증가하여 입력크기와 비슷해 질수록, 예상하듯 더 넓은 범위의 정보를 얻어내는 것이 아니라 단순한 1×1 필터를 이용한 효과와 비슷해지는 것을

확인할 수 있다. 이는 3×3 필터의 가중치 중 가운데 가중치만 의미있는 연산을 진행하기 때문이다. 이와 같은 문제를 극복하고 넓은 범위의 쓸모있는 정보를 얻어내기 위해서는 다음과 같은 절차를 거친다.

우선, global average pooling을 진행한다. pooling의 결과를 가지고 256의 필터로 이루어진 1×1 convolution을 진행한다. 이때, batch normalization도 진행한다. 이 결과값(result)을 바탕으로 마지막으로 하나의 1×1 convolution과 각각 6, 12, 18의 atrous rate를 가진 (output stride가 16일 때) convolution을 병렬로 진행한다. 이후 각 branch들의 결과와 (result)를 concat한 후 1×1 convolution을 진행하여 마지막 logit을 산출해낸다.

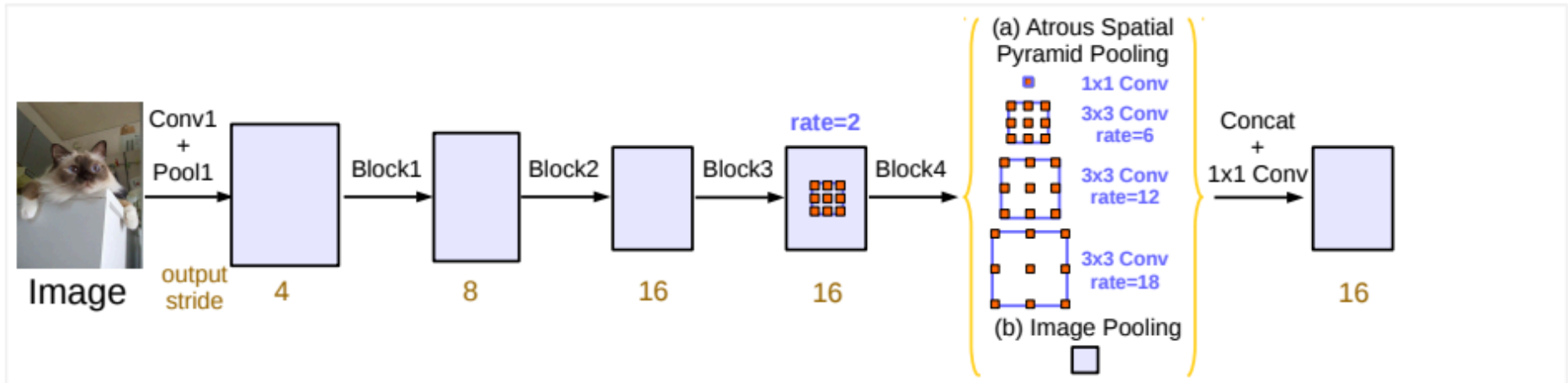


Figure 5. Parallel modules with atrous convolution (ASPP), augmented with image-level features.